



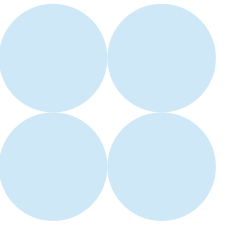
OBJECT ORIENTED PROGRAMMING (OOP)

BASIC CONCEPT OF OOP

- OOP TEAM-

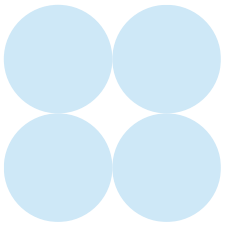


Outline



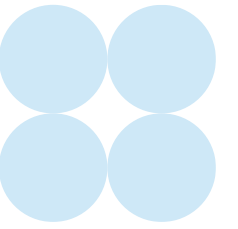
- Syllabus
- Procedural vs. Object-Oriented
- Basic Concepts of Object-Oriented Programming
- UML Class Diagram

Mata kuliah PBO



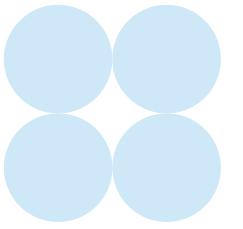
- Object-Oriented Programming (2 credits/3 hours)
- Object-Oriented Programming Lab (3 credits/6 hours)
- Learning Outcomes
 - Be able to create programs using OOP principles in the Java programming language
- Software:
 - JDK (Java Development Kit)
 - NetBeans/Visual Studio Code

Syllabus



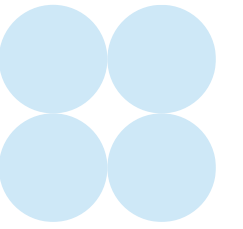
- Introduction to Basic OOP Concepts
- Classes and Objects
- Encapsulation
- Quiz 1
- Class Relationships
- Inheritance
- Overriding and Overloading
- Midterm Exam
- Abstract Classes and Interfaces
- Polymorphism
- GUI
- Quiz 2
- GUI and Databases
- GUI and Databases
- Project
- Project
- Final Exam

Procedural vs. Object-Oriented



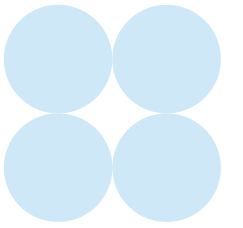
- Procedural
 - The program is broken down into functions
 - Feature changes may disrupt the entire program
- Object-Oriented
 - The program is broken down into objects
 - These objects contain state and behavior
 - Feature changes do not disrupt the entire program

Procedural vs. Object-Oriented



- Example:
 - We're going to create a bicycle simulation game, featuring a bicycle character with a speed and brand.
 - How do we build this game using conventional methods?
 - The first step is to declare the variables, such as speed and brand.
 - The next step is to create the functions: increase speed and decrease speed.
 - The next step is to try operating the bicycle in a simple way—that is, manipulating its speed and brand within the main function—and then print the results to the screen.

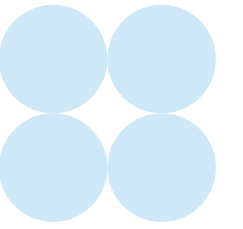
Procedural vs. Object-Oriented



J StructuralBike.java > ...

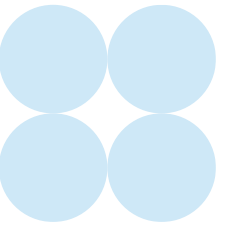
```
1  public class StructuralBike {  
    Run | Debug  
2      public static void main(String[] args) {  
3          String brand;  
4          int speed, speedChange;  
5  
6          brand = "Trek";  
7          speed = 10;  
8          speedChange = 5;  
9  
10         speed = speedAcceleration(speed, speedChange);  
11  
12         System.out.println("Brand: " + brand);  
13         System.out.println("Speed: " + speed);  
14     }  
15  
16     public static int speedAcceleration(int speed, int increment) {  
17         speed += increment;  
18         return speed;  
19     }  
20  
21     public static int speedDeceleration(int speed, int decrement) {  
22         speed -= decrement;  
23         return speed;  
24     }  
25 }
```

Procedural vs. Object-Oriented



- What if there are two bikes in the game?
 - Add the variables brand2, speedParam2, and speedChange2.
 - Try manipulating the values of these variables and then display them on the screen.
- Program code → ...

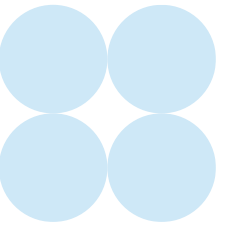
Procedural vs. Object-Oriented



J StructuralBike2.java > ...

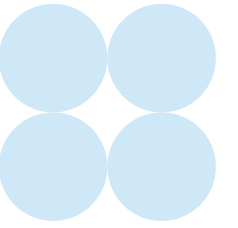
```
1  public class StructuralBike2 {  
    Run | Debug  
2      public static void main(String[] args) {  
3          String brand;  
4          int speed, speedChange, speed2, speedChange2;  
5  
6          brand = "Trek";  
7          speed = 10;  
8          speedChange = 5;  
9  
10         speed2 = 15;  
11         speedChange2 = 3;  
12  
13         speed = speedAcceleration(speed, speedChange);  
14         speed2 = speedAcceleration(speed2, speedChange2);  
15  
16         System.out.println("Brand: " + brand);  
17         System.out.println("Speed: " + speed);  
18         System.out.println("Speed2: " + speed2);  
19     }  
20  
21     public static int speedAcceleration(int speed, int increment) {  
22         speed += increment;  
23         return speed;  
24     }  
25  
26     public static int speedDeceleration(int speed, int decrement) {  
27         speed -= decrement;  
28         return speed;  
29     }  
30 }
```

Procedural vs. Object-Oriented



- What if there are ten bikes?
 - Add the variables brand3, speed3, brand9, and speed9.
 - That's pretty tedious...
- What about object-oriented programming?
 - Create a Bike class with the attributes brand and speed.
 - Create 10 bike objects.
- Program code →...

Object-Oriented Programming



```
1 package ppt;
2 class Bike{
3     private String brand;
4     private int speed;
5
6     public Bike(String brand, int speed) {
7         this.brand = brand;
8         this.speed = speed;
9     }
10
11     public String getBrand() {
12         return brand;
13     }
14
15     public int getSpeed() {
16         return speed;
17     }
18
19     public void setSpeed(int speed) {
20         this.speed = speed;
21     }
22
23     public int speedAcceleration(int increment) {
24         speed += increment;
25         return speed;
26     }
27
28     public int speedDeceleration(int decrement) {
29         speed -= decrement;
30         return speed;
31     }
32 }
```

```
public class Bike00p {
```

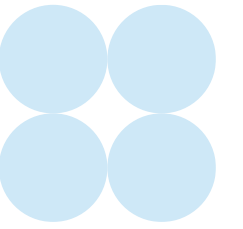
Run | Debug

```
public static void main(String[] args) {
    Bike bike1 = new Bike("Trek", 10);
    bike1.speedAcceleration(5);

    Bike bike2 = new Bike("United", 5);
    bike2.speedAcceleration(10);

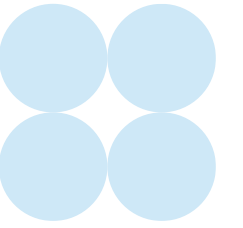
    System.out.println("Brand Bike1: " + bike1.getBrand());
    System.out.println("Speed Bike1: " + bike1.getSpeed());
    System.out.println("Brand Bike2: " + bike2.getBrand());
    System.out.println("Speed Bike2: " + bike2.getSpeed());
}
```

Procedural vs. Object-Oriented



- As we can see, with OOP, to create multiple bikes, we don't need to repeatedly write the brand and speed variables.
- We simply create multiple bike objects from the Bike class we've already created.

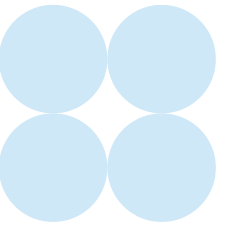
Basic Concepts of PBO



- Some aspects of OOP:

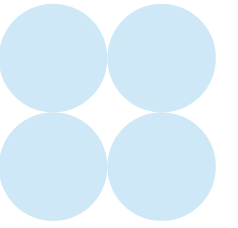
- Object
- Class
- Encapsulation
- Inheritance
- Polymorphism

Object



- An object is a component of a program consisting of state and behavior.
- Objects in software are modeled in such a way that they resemble objects in the real world.
- Objects have state and behavior.
- State refers to the characteristics or attributes of the object.
- For example, a bicycle object has states such as brand, speed, gear, and so on.
- Behavior refers to the actions the object can perform.
- For example, a bicycle's behaviors include accelerating, decelerating, turning, and so on.

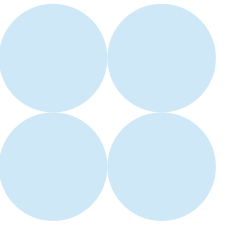
Class



- A class is a blueprint or prototype of an object.
- Take the bicycle as an example.
- There are many different types of bicycles in the world, from various brands and models.
- However, all bicycles are built based on the same blueprint, so each bicycle has the same components and characteristics.
- The bicycle you have at home is an instance of the Bicycle class.



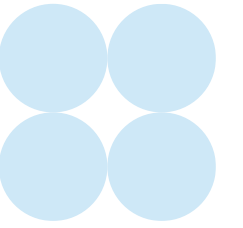
Encapsulation



- This is also known as information hiding.
 - When interacting with an object, we often don't need to know the inner workings of that object.
- Take a bicycle as an example:
 - When we shift gears on a bicycle, all we have to do is press the gear lever on the handlebar grips.
 - We don't need to know the technical details of how the gears shift.

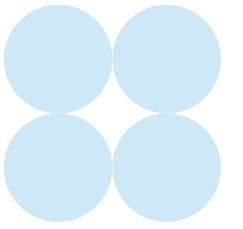


Inheritance



- It allows us to organize the program's structure naturally.
- It expands the program's functionality without having to change many parts of the program.
- Real-world example:
 - The “bicycle” object can be further subdivided into broader categories, such as mountain bikes and city bikes.
 - Each can have additional components or features; for example, mountain bikes have suspension, which regular bikes do not have, and city bikes have a basket on the front.
 - In this case, the “mountain bike” and “city bike” objects inherit from the “bicycle” object.

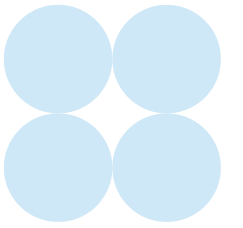
Polymorphism



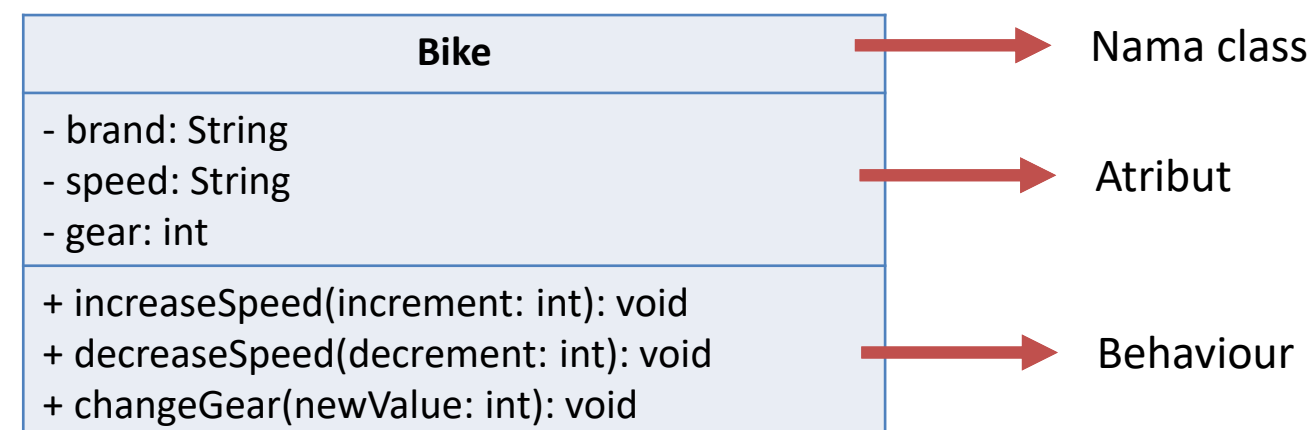
- Polymorphism also mimics the properties of objects in the real world, where an object can take on different forms or transform into other forms.
- for example, an airplane.
- This object can be instantiated as a jet or a propeller plane.
- Both have the ability to increase speed.
- However, technically, the methods for increasing speed differ between a jet and a propeller plane, since each has a different type of engine.



UML Class Diagram



- In object-oriented programming, class designs are represented using UML class diagrams.
- UML stands for Unified Modeling Language.
- For example, the `Bicycle` class—which has the states `brand`, `speed`, and `gear`, and the behaviors `increaseSpeed`, `decreaseSpeed`, and `changeGear`—is represented in a class diagram as follows:

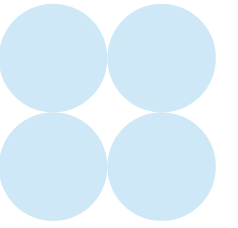




THANK YOU

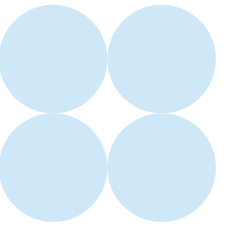


Exercise



- Find 5 objects from the real world.
- Describe the states and behaviors of those objects. The more states and behaviors, the better. Example:
- Television
 - State:
 - Brand
 - Screen Size
 - Channels
 - Volume
 - Behavior:
 - Turn on
 - Turn off
 - Change channel
 - Turn up the volume
 - Turn down the volume

Exercise



1. An object consists of and
2. In a program, the state of an object is stored in
3. In a program, the behavior of an object is defined by
4. Hiding the complexity within an object is called
5. The blueprint of an object is called
6. Explain what inheritance means and give a real-world example!